A

Microproject

On

Tic Tac Toe Game

Submitted To

MSBTE

In Partial Fulfilment of Requirement of

Diploma of Computer Engineering

Under I Scheme

Submitted By

Mr.Aniket Nilkakanth Gawas

Mr.Narayan Rajan Dinganekar

Mr.Vasant Satyawan Gawade

Mr.Ayush Arvind Desai

Under the Guidance Of

Ms.T.V.Gawandi

FOR ACADEMIC YEAR 2023-2024

YASHWANTRAO BHONSALE INSTITUTE OF,

TECHNOLOGY,SAWANTWADI.

MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION

CERTIFICATE

This is to certify that: -

Mr.Aniket Nilkanth Gawas	Roll No.07
Mr.Narayan Rajan Dinganekar	Roll No.08
Mr.Vasant Satyawan Gawade	Roll No.11
Mr.Ayush Arvind Desai	Roll No.15

Of **Sixth** Semester of diploma in **COMPUTER ENGINEERING** Of institute **Yashwantrao Bhonsale Institute Of Technology (1742)** has completed the **Micro Microproject** satisfactorily in subject **Mobile Application Development (22617)** for the academic year 2023 to 2024 as prescribed in the curriculum.

……	……...	…….....
Subject Teacher	Head of Department	Principal

Seal of Institution

INDEX

Abstract

Tic-tac-toe (American English), noughts and crosses (Commonwealth English), or Xs and Os (Canadian or Irish English) is a paper-and-pencil game for two players who take turns marking the spaces in a three-by-three grid with X or O. The player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal row is the winner. It is a solved game, with a forced draw assuming best play from both players.

Introduction

Tic-tac-toe is played on a three-by-three grid by two players, who alternately place the marks X and O in one of the nine spaces in the grid.

In the following example, the first player (X) wins the game in seven steps:

Fig 1.Tic Tac Toe Structure

There is no universally-agreed rule as to who plays first, but in this article the convention that X plays first is used.

Players soon discover that the best play from both parties leads to a draw. Hence, tic-tac-toe is often played by young children who may not have discovered the optimal strategy. Because of the simplicity of tic-tac-toe, it is often used as a pedagogical tool for teaching the concepts of good sportsmanship and the branch of artificial intelligence that deals with the searching of game trees.

It is one of the most popular two-player pen-and-paper games. This two-player pen-and-paper game has extremely ancient roots and is popular in many countries. The game's rules are straightforward and well-known. Two players, X and O, will play this game by alternately marking the squares in a 3X3 grid. The game is won by the person who successfully arranges three of their marks in a row that is either horizontal, vertical, or diagonal. Only one symbol may be placed by each player at every turn, after which the turn is passed to the other player.

Game Approach And Specific Aspects

Tic-tac-toe is an ideal educational game for teaching youngsters the development of logic and sportsmanship because of its simplicity.

A n x n version of the traditional 3 x 3 game of tic-tac-toe may be created in which the two players alternately put their symbols on a n × n board to line up d of their symbols in a row that is either vertical, horizontal, or diagonal. In this blog, we'll use Java programming to create a game Tic Tac Toe console version. The fundamental concept is to manage the game board using a two-dimensional array or board. This array's cells contain values that specify whether they are empty or have an X or an O.

Consider a 3 x 3 board so the player board will look like the image below.

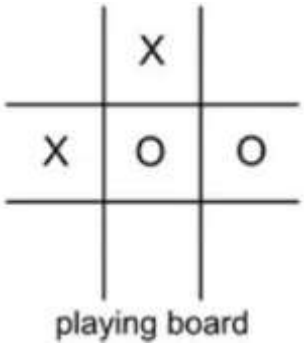


Fig 2.Tic Tac Toe board

And the board array will look something like the below image.

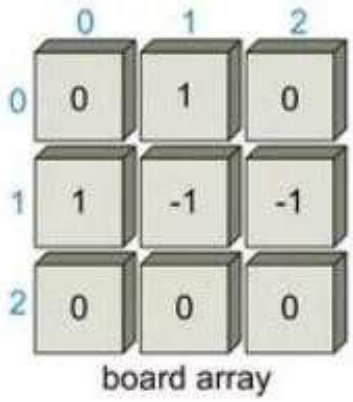


Fig 3.Two-Dimensional board array

The center row of the 3 x 3 matrix board has the cells board[1][0], board[1][1], and board[1][2], where the first number represents the row number and the second number represents the column number of the board.

Specific aspects to consider about the game in Java

- The program outputs a 3×3 board with dashes, which stand for vacant spaces.
- The board is again printed with the x or o in the appropriate position after each turn by asking either player 1 or player 2 to input a row and col index representing where they wish to place their x and o.
- Our application alerts the player and asks them to enter another row and col if the place they entered is “off the board” or already contains an x or o.
- The program prints out the final board and notifies players 1 or 2 that they have won after they obtain 3 symbols in a row, column, or diagonal.
- **Wins**: Each player attempts to arrange three symbols in three adjacent cells that are either horizontally, vertically, or diagonally spaced. The winner is the one who accomplishes this alignment first. The second player attempts to obstruct Player 1’s alignment by inserting his symbols between Player 1’s symbols.
- **Lose**: You lose if your rival achieves the necessary symbol alignment faster.
- **Draw**: If none of the players achieves the necessary alignment, all nine grip cells are marked.. A draw or tie is the outcome. In this scenario, none of the players receives a point. Throughout the game, this circumstance occurs frequently and is quite entertaining.

General Implementation Order

- Dashes should be placed on a Tic Tac Toe board.
- Create a function that squares up the board.
- The player's turn and the symbol they are using should be noted.
- Ask for a row and a col repeatedly until they are appropriate.
- Set the appropriate symbol in the relevant place on the board.
- Make a function that determines who won the game.
- Test to see if there was a tie in the game.
- To keep the game alive, use a loop.

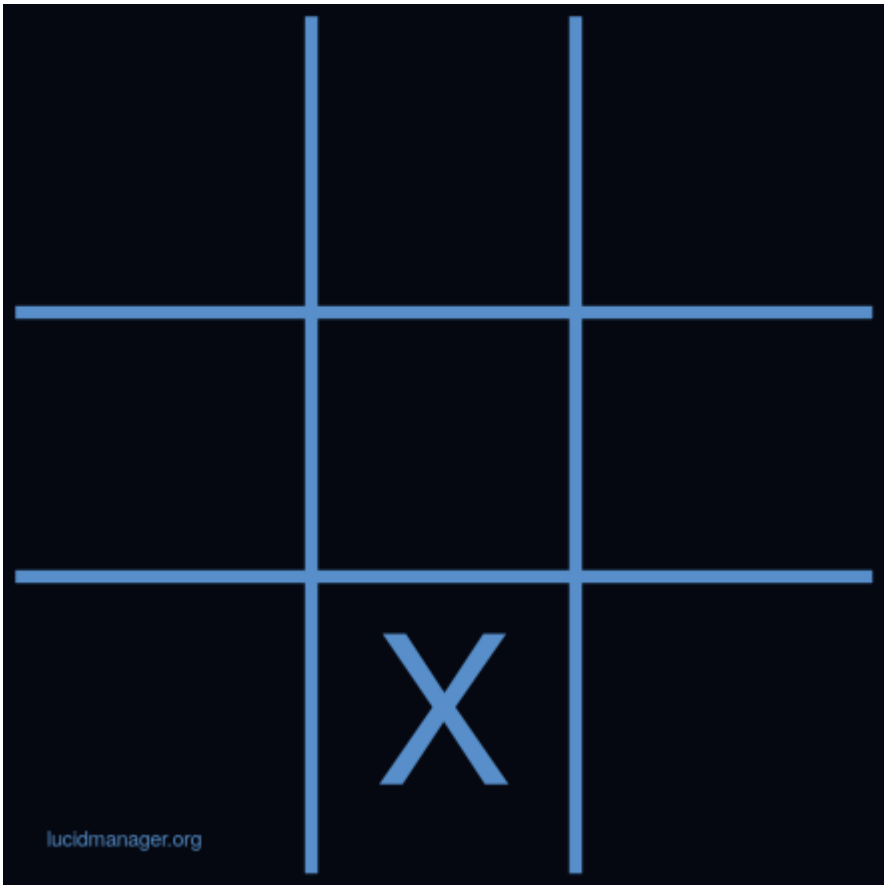


Fig 4.Tic Tac Toe Implementation

Algorithm

Step I : Start

Step II : To simulate the tic tac toe board, make a 3×3 array and fill it with dashes

This line of code is used to create a 3×3 character array. Each point on the board is iterated through using a nested for loop. We may put `board[i][j]` equal to a dash within both for loops.

Step III: Make a function that prints the board as a 3×3 square after the board is drawn

For the function to be able to print the board 2D array, we must supply it. Since the function only prints the board, we don't need to return anything.We need to print out each place on our board inside of our function.

Step IV: Make a function that prints the board as a 3×3 square after the board is drawn

In our game, we need a means to keep track of which player is taking turns. We will use a boolean variable named player1 that is true when player 1 is doing the turn and false when player 2 is.

Step V: Ask for a row and a col repeatedly until they are appropriate

Use the `Scanner` to obtain the user's input and store it in a variable named row after printing a message asking for a row; do the same for a variable called col. The row and col are invalid if the user enters them in a position that is not on the board. To determine whether the row and col are not larger than 2 and not less than 0, use a conditional. The row and col are invalid if the user enters them in a cell that already has an x or an o on it. Check to see if the board location at row and col already has an x or o by using a conditional. We want to prompt the user to re-enter a row and col if they input a row and col that is outside of bounds or that already has an x or o on it. For this, we will use the while loop which will work great! And it will break out of the loop once the player has entered a valid row and col.

Step VI: Set the appropriate symbol in the relevant place on the board

We are clear that we have an appropriate row and col outside of the while loop.By doing `board[row][col]`, we may obtain the desired location on the board.Now that we have put the player's character in the variable c, we can set this position to equal that value.

Step VII: Make a function that determines who won the game

A player in the game of tic tac toe wins if they line up three of their symbols in a row, column, or diagonal.Start off with rows. To iterate through each row we may use a for loop.We may use a conditional within the for loop to determine if `board[i][0]` equals `board[i][1]` and whether `board[i][1]` equals `board[i][2]`. Remember that in order to prevent winning if there are three consecutive empty places, we must additionally verify that `board[i][0]` does not equal a

dash. We can return the value of `board[i][0]` if everything is true.Similar procedures may be used for columns.On the board, there are two diagonals that need to be examined. Similar to the if statements we used for rows and columns, we can use two if statements to verify the two diagonals.When our function is complete, it indicates that no one has won. Then let's return space in this case.

Step VIII: Test to see if there was a tie in the game

A player in the game of tic tac toe wins if they line up three of their symbols in a row, column, or diagonal.Start off with rows. To iterate through each row we may use a for loop.We may use a conditional within the for loop to determine if `board[i][0]` equals `board[i][1]` and whether `board[i][1]` equals `board[i][2]`. Remember that in order to prevent winning if there are three consecutive empty places, we must additionally verify that `board[i][0]` does not equal a dash. We can return the value of `board[i][0]` if everything is true.Similar procedures may be used for columns.On the board, there are two diagonals that need to be examined. Similar to the if statements we used for rows and columns, we can use two if statements to verify the two diagonals.When our function is complete, it indicates that no one has won. Then let's return space in this case.

Step IX: To keep the game alive, use a loop

We may make a boolean named gameEnded and set it to false at the beginning. We may set gameEnded to true inside the if statement, which is where we determine if a player has won or whether it is a tie.In order for the program to continue requesting a player to enter a row and a column until a winner or a tie is determined, we may create a while loop with the condition simply being gameEnded.We can draw the board one last time once the while loop is finished so that both players can view the board's final configuration.

Step X: Stop

Program

XML Code:-

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/green"
    tools:context=".MainActivity">

    <!--title text-->
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="23dp"
        android:text="GFG Tic Tac Toe"
        android:textSize="45sp"
        android:textStyle="bold"
        app:fontFamily="cursive"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <!--image of the grid-->
    <ImageView
        android:id="@+id/imageView"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:contentDescription="Start"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
```

```
        app:layout_constraintTop_toBottomOf="@+id/textView"
        app:srcCompat="@drawable/grid" />

    <LinearLayout
        android:id="@+id/linearLayout"
        android:layout_width="0dp"
        android:layout_height="420dp"
        android:orientation="vertical"
        app:layout_constraintBottom_toBottomOf="@+id/imageView"
        app:layout_constraintEnd_toEndOf="@+id/imageView"
        app:layout_constraintStart_toStartOf="@+id/imageView"
        app:layout_constraintTop_toTopOf="@+id/imageView">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:orientation="horizontal">

            <!--images of the grid boxes-->
            <ImageView
                android:id="@+id/imageView0"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:onClick="playerTap"
                android:padding="20sp"
                android:tag="0" />

            <ImageView
                android:id="@+id/imageView1"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:onClick="playerTap"
```

```
                android:padding="20sp"
                android:tag="1" />

        <ImageView
                android:id="@+id/imageView2"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:onClick="playerTap"
                android:padding="20sp"
                android:tag="2" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:orientation="horizontal">

        <ImageView
                android:id="@+id/imageView3"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:onClick="playerTap"
                android:padding="20sp"
                android:tag="3" />

        <ImageView
                android:id="@+id/imageView4"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:onClick="playerTap"
                android:padding="20sp"
```

```
                android:tag="4" />

        <ImageView
                android:id="@+id/imageView5"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:onClick="playerTap"
                android:padding="20sp"
                android:tag="5" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:orientation="horizontal">

        <ImageView
                android:id="@+id/imageView6"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:onClick="playerTap"
                android:padding="20sp"
                android:tag="6" />

        <ImageView
                android:id="@+id/imageView7"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:onClick="playerTap"
                android:padding="20sp"
                android:tag="7" />
```

```
                <ImageView
                        android:id="@+id/imageView8"
                        android:layout_width="match_parent"
                        android:layout_height="match_parent"
                        android:layout_weight="1"
                        android:onClick="playerTap"
                        android:padding="20sp"
                        android:tag="8" />
        </LinearLayout>

    </LinearLayout>

    <!--game status text display-->
    <TextView
        android:id="@+id/status"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="15sp"
        android:text="Status"
        android:textSize="28sp"
        android:textStyle="italic"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/linearLayout" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Java Code:-

```
import android.os.Bundle;
import android.view.View;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    boolean gameActive = true;

    // Player representation
    // 0 - X
    // 1 - O
    int activePlayer = 0;
    int[] gameState = {2, 2, 2, 2, 2, 2, 2, 2, 2};

    // State meanings:
    // 0 - X
    // 1 - O
    // 2 - Null
    // put all win positions in a 2D array
    int[][] winPositions = {{0, 1, 2}, {3, 4, 5}, {6, 7, 8},
                {0, 3, 6}, {1, 4, 7}, {2, 5, 8},
                {0, 4, 8}, {2, 4, 6}};
    public static int counter = 0;

    // this function will be called every time a
    // players tap in an empty box of the grid
    public void playerTap(View view) {
        ImageView img = (ImageView) view;
        int tappedImage = Integer.parseInt(img.getTag().toString());
```

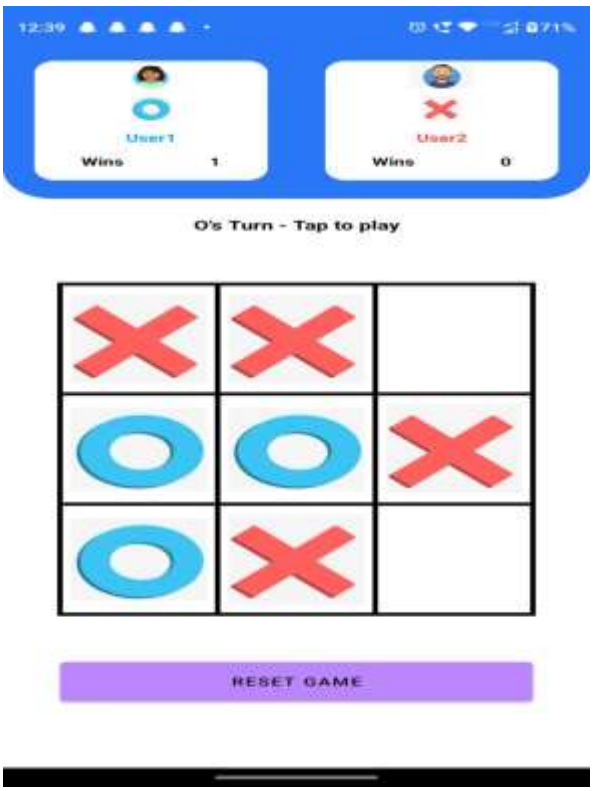
```
// game reset function will be called
// if someone wins or the boxes are full
if (!gameActive) {
        gameReset(view);
        //Reset the counter
        counter = 0;
}

// if the tapped image is empty
if (gameState[tappedImage] == 2) {
        // increase the counter
        // after every tap
        counter++;

        // check if its the last box
        if (counter == 9) {
                // reset the game
                gameActive = false;
        }

        // mark this position
        gameState[tappedImage] = activePlayer;

        // this will give a motion
        // effect to the image
        img.setTranslationY(-1000f);

        // change the active player
        // from 0 to 1 or 1 to 0
        if (activePlayer == 0) {
                // set the image of x
```

```
                img.setImageResource(R.drawable.x);
                activePlayer = 1;
                TextView status = findViewById(R.id.status);

                // change the status
                status.setText("O's Turn - Tap to play");
            } else {
                // set the image of o
                img.setImageResource(R.drawable.o);
                activePlayer = 0;
                TextView status = findViewById(R.id.status);

                // change the status
                status.setText("X's Turn - Tap to play");
            }
            img.animate().translationYBy(1000f).setDuration(300);
        }
        int flag = 0;
        // Check if any player has won if counter is > 4 as min 5 taps are
        // required to declare a winner
        if (counter > 4) {
        for (int[] winPosition : winPositions) {
            if (gameState[winPosition[0]] == gameState[winPosition[1]] &&
                    gameState[winPosition[1]] == gameState[winPosition[2]] &&
                    gameState[winPosition[0]] != 2) {
                flag = 1;

                // Somebody has won! - Find out who!
                String winnerStr;

                // game reset function be called
                gameActive = false;
```

```
                    if (gameState[winPosition[0]] == 0) {
                        winnerStr = "X has won";
                    } else {
                        winnerStr = "O has won";
                    }
                    // Update the status bar for winner announcement
                    TextView status = findViewById(R.id.status);
                    status.setText(winnerStr);
                }
            }
            // set the status if the match draw
            if (counter == 9 && flag == 0) {
                TextView status = findViewById(R.id.status);
                status.setText("Match Draw");
            }
            }
        }

        // reset the game
        public void gameReset(View view) {
            gameActive = true;
            activePlayer = 0;
            //set all position to Null
            Arrays.fill(gameState, 2);
            // remove all the images from the boxes inside the grid
            ((ImageView) findViewById(R.id.imageView0)).setImageResource(0);
            ((ImageView) findViewById(R.id.imageView1)).setImageResource(0);
            ((ImageView) findViewById(R.id.imageView2)).setImageResource(0);
            ((ImageView) findViewById(R.id.imageView3)).setImageResource(0);
            ((ImageView) findViewById(R.id.imageView4)).setImageResource(0);
            ((ImageView) findViewById(R.id.imageView5)).setImageResource(0);
            ((ImageView) findViewById(R.id.imageView6)).setImageResource(0);
```

```
        ((ImageView) findViewById(R.id.imageView7)).setImageResource(0);
        ((ImageView) findViewById(R.id.imageView8)).setImageResource(0);

        TextView status = findViewById(R.id.status);
        status.setText("X's Turn - Tap to play");
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Output Of Micro-Project:



Conclusion:-

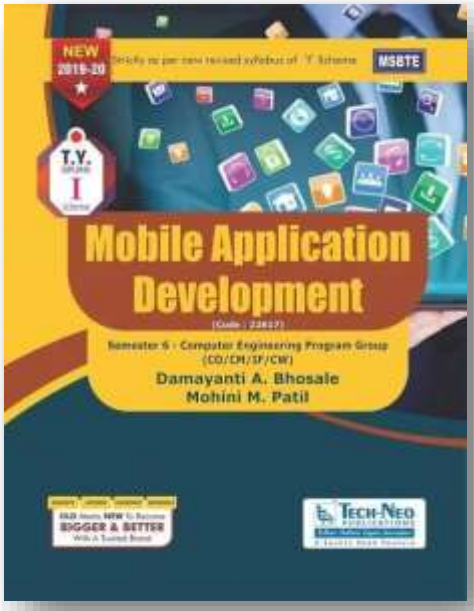
In this Microproject, we learned how to implement the fundamental tic-tac-toe game in Mobile Application development. We used common modules/tags in a Tic Tac Toe mobile app UI components for the game board and buttons, game logic for win conditions and player turns, event handling for user input, and optional features like graphics/animations, sound effects, and data storage for game state Learning to code can give your child the foundational skills for future success. Basic programming knowledge can change the way kids interact with the technologies they use daily. It's a basic literacy– one we can't afford to overlook. We learnt so many thing and gained a lot of knowledge about development field.

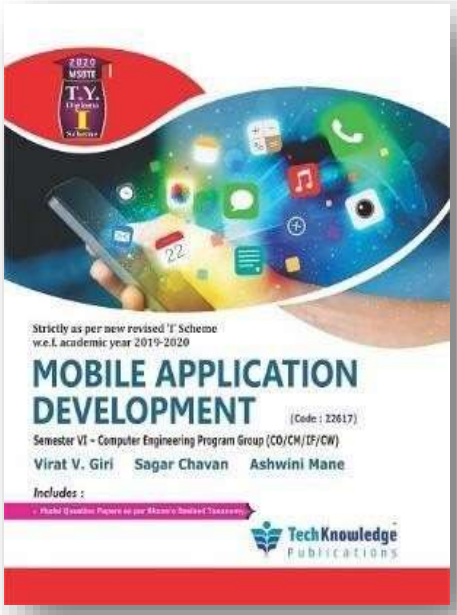
Reference

We Referred Some Website :

- https://www.geeksforgeeks.org/how-to-build-a-tic-tac-toe-game-in-android/
- https://en.wikipedia.org/wiki/Tic-tac-toe

We Referred Some Book :





Referred Some You-Tube Video :

- https://www.youtube.com/watch?v=M6QuAMC2xXkhttps://www.youtube.com/watch?v=M6QuAMC2xXk
- https://www.youtube.com/watch?v=otHIBvufOrc&list=RDCMUCvT21bzLRHrnAc-F1SRqRSw&index=3